Tennis Rankings Explorer: Project Documentation

Game Analytics: Unlocking Tennis Data with SportRadar API

Author: Dhamayanthi Devi K | GitHub:
https://github.com/dhamayanthidevik/tennis_project

1. Project Overview

Tennis Rankings Explorer is an end-to-end data analytics project. It collects tennis data from the SportRadar Tennis API, cleans and stores it in a relational MySQL database, analyses it with SQL, and presents the results in an interactive Streamlit web application.

The app lets users explore doubles rankings, countries, competitions and venues through seven pages with filters, tables and charts.

Domain: Sports / Data Analytics

Skills demonstrated: Python scripting, API integration, JSON parsing, data cleaning with Pandas, relational database design, SQL analysis, Streamlit dashboard development, GitHub documentation and cloud deployment.

2. Objectives and Business Use Cases

Objectives

- Collect competition, complex (venue) and doubles competitor ranking data from the SportRadar Tennis API.
- Transform nested JSON into a flat relational schema with primary and foreign keys.
- Run SQL queries to answer analytical questions.
- Build an interactive dashboard connected to the database.

Business use cases

1. **Event exploration:** navigate competition hierarchies, from parent competitions to sub-competitions.
2. **Trend analysis:** view the distribution of competitions by type, gender and category.
3. **Performance insights:** compare competitors, countries and ranking points.

4. **Decision support:** give event organisers and sports bodies data to support planning and resource allocation.

3. Tech Stack

Area	Tools
Language	Python
Data collection	SportRadar Tennis API, requests
Data processing	Pandas, JSON
Database	MySQL (local MySQL Workbench, then hosted on Aiven)
Web app	Streamlit
Version control and hosting	Git, GitHub, Streamlit Cloud

4. Project Workflow

Step	File	Purpose
1	`downloadjson.py`	Calls the API endpoints and saves raw JSON (competitions, complexes, doubles rankings)
2	`preprocessing.py`	Cleans and flattens the raw JSON
3	`Build_table.py`	Builds one CSV per table (6 tables)
4	`Create_query.sql`	Creates the tables with primary and foreign key constraints
5	`loadmysql.py`	Loads the CSV data into MySQL
6	`analysis_queries.sql`	SQL queries used for analysis and the SQL Insights page
7	`app/`	Streamlit application

5. Data Collection

Data was collected from three SportRadar Tennis API feeds:

1. **Competitions:** competition id, name, parent id, type, gender and category.
2. **Complexes:** complexes and the venues inside them, with city, country, country code and timezone.
3. **Doubles competitor rankings:** rank, movement, points, competitions played and competitor details.

Handling API rate limits: the trial key has request limits, so small delays were added between API calls, and raw responses were saved to `data/raw` so the API did not need to be called again during cleaning and testing.

6. Database Design

The database `tennis_db` contains six tables.

Categories

Column	Type	Constraint
category_id	VARCHAR(50)	PRIMARY KEY
category_name	VARCHAR(100)	NOT NULL

Competitions

Column	Type	Constraint
competition_id	VARCHAR(50)	PRIMARY KEY
competition_name	VARCHAR(100)	NOT NULL
parent_id	VARCHAR(50)	NULLABLE
type	VARCHAR(20)	NOT NULL
gender	VARCHAR(10)	NOT NULL
category_id	VARCHAR(50)	FOREIGN KEY to Categories

Complexes

Column	Type	Constraint
complex_id	VARCHAR(50)	PRIMARY KEY
complex_name	VARCHAR(100)	NOT NULL

Venues

Column	Type	Constraint
venue_id	VARCHAR(50)	PRIMARY KEY
venue_name	VARCHAR(100)	NOT NULL
city_name	VARCHAR(100)	NOT NULL
country_name	VARCHAR(100)	NOT NULL
country_code	CHAR(3)	NOT NULL
timezone	VARCHAR(100)	NOT NULL
complex_id	VARCHAR(50)	FOREIGN KEY to Complexes

Competitors

Column	Type	Constraint
competitor_id	VARCHAR(50)	PRIMARY KEY
name	VARCHAR(100)	NOT NULL
country	VARCHAR(100)	NOT NULL
country_code	CHAR(3)	NOT NULL
abbreviation	VARCHAR(10)	NOT NULL

Competitor_Rankings

Column	Type	Constraint
rank_id	INT	PRIMARY KEY, AUTO_INCREMENT
rank	INT	NOT NULL
movement	INT	NOT NULL
points	INT	NOT NULL
competitions_played	INT	NOT NULL
competitor_id	VARCHAR(50)	FOREIGN KEY to Competitors
tour, gender, year, week	various	Extra columns used for filtering in the app

Relationships

- One category has many competitions (Categories to Competitions).
- One complex has many venues (Complexes to Venues).
- One competitor has many ranking records (Competitors to Competitor_Rankings).
- Competitions link to themselves through `parent_id`, which forms the competition hierarchy.

7. SQL Analysis

Twenty SQL queries were written across the three data groups. Note that `rank` is a reserved word in MySQL 8, so it is written with backticks.

7.1 Competitions and categories

```
-- 1. All competitions with their category name
SELECT c.competition_id, c.competition_name, cat.category_name
FROM competitions c
JOIN categories cat ON c.category_id = cat.category_id;

-- 2. Number of competitions in each category
SELECT cat.category_name, COUNT(*) AS total_competitions
FROM competitions c
JOIN categories cat ON c.category_id = cat.category_id
```

```
GROUP BY cat.category_name
ORDER BY total_competitions DESC;

-- 3. All competitions of type 'doubles'
SELECT * FROM competitions WHERE type = 'doubles';

-- 4. Competitions in a specific category (e.g., ITF Men)
SELECT c.competition_name, cat.category_name
FROM competitions c
JOIN categories cat ON c.category_id = cat.category_id
WHERE cat.category_name = 'ITF Men';

-- 5. Parent competitions and their sub-competitions
SELECT p.competition_name AS parent_competition,
       c.competition_name AS sub_competition
FROM competitions c
JOIN competitions p ON c.parent_id = p.competition_id;

-- 6. Distribution of competition types by category
SELECT cat.category_name, c.type, COUNT(*) AS total
FROM competitions c
JOIN categories cat ON c.category_id = cat.category_id
GROUP BY cat.category_name, c.type
ORDER BY cat.category_name, total DESC;

-- 7. Top-level competitions (no parent)
SELECT * FROM competitions WHERE parent_id IS NULL;
```

7.2 Complexes and venues

```
-- 1. All venues with their complex name
SELECT v.venue_name, cx.complex_name
FROM venues v
JOIN complexes cx ON v.complex_id = cx.complex_id;

-- 2. Number of venues in each complex
SELECT cx.complex_name, COUNT(*) AS total_venues
FROM venues v
JOIN complexes cx ON v.complex_id = cx.complex_id
GROUP BY cx.complex_name
ORDER BY total_venues DESC;
```

```
-- 3. Venues in a specific country (e.g., Chile)
SELECT * FROM venues WHERE country_name = 'Chile';

-- 4. All venues and their timezones
SELECT venue_name, timezone FROM venues;

-- 5. Complexes with more than one venue
SELECT cx.complex_name, COUNT(*) AS total_venues
FROM venues v
JOIN complexes cx ON v.complex_id = cx.complex_id
GROUP BY cx.complex_name
HAVING COUNT(*) > 1;

-- 6. Venues grouped by country
SELECT country_name, COUNT(*) AS total_venues,
       GROUP_CONCAT(venue_name SEPARATOR ', ') AS venues
FROM venues
GROUP BY country_name;

-- 7. All venues for a specific complex (e.g., Nacional)
SELECT v.*
FROM venues v
JOIN complexes cx ON v.complex_id = cx.complex_id
WHERE cx.complex_name = 'Nacional';
```

7.3 Competitors and rankings

```
-- 1. All competitors with rank and points
SELECT c.name, r.`rank`, r.points
FROM competitors c
JOIN competitor_rankings r ON c.competitor_id = r.competitor_id;

-- 2. Competitors ranked in the top 5
SELECT c.name, c.country, r.`rank`, r.points
FROM competitors c
JOIN competitor_rankings r ON c.competitor_id = r.competitor_id
WHERE r.`rank` <= 5
ORDER BY r.`rank`;

-- 3. Competitors with no rank movement (stable rank)
SELECT c.name, r.`rank`, r.movement
FROM competitors c
```

```
JOIN competitor_rankings r ON c.competitor_id = r.competitor_id
WHERE r.movement = 0;

-- 4. Total points of competitors from a specific country (e.g., Croatia)
SELECT c.country, SUM(r.points) AS total_points
FROM competitors c
JOIN competitor_rankings r ON c.competitor_id = r.competitor_id
WHERE c.country = 'Croatia'
GROUP BY c.country;

-- 5. Number of competitors per country
SELECT country, COUNT(*) AS total_competitors
FROM competitors
GROUP BY country
ORDER BY total_competitors DESC;

-- 6. Competitors with the highest points in the current (latest) week
SELECT c.name, c.country, r.points
FROM competitors c
JOIN competitor_rankings r ON c.competitor_id = r.competitor_id
WHERE (r.year, r.week) = (
    SELECT year, week FROM competitor_rankings
    ORDER BY year DESC, week DESC LIMIT 1
)
ORDER BY r.points DESC
LIMIT 10;
```

Please compare these with your `analysis_queries.sql` file and keep the final version in the repo consistent with this section.

8. Streamlit Application

The app connects directly to the SQL database and runs queries in real time. Navigation is a sidebar radio menu with seven pages.

Page	What it shows
Home Dashboard	Summary cards (total competitors, competitions, venues, countries), top 10 countries chart, competitions by type chart, top 5 ranked competitors

Page	What it shows
Competitor Explorer	Search competitors by name and filter by rank range, country and points
Competitor Details	Rank, movement, competitions played and country for one selected competitor
Country Analysis	Number of competitors and average points by country
Leaderboards	Top ranked competitors and competitors with the highest points
Competitions & Venues	Browse competitions by type, gender and category, and venues by country and complex
SQL Insights	Results of the SQL analysis queries

Screenshots of every page are in the `screenshots/` folder of the repository and in the README.

9. Key Insights

From the Home Dashboard:

- The database holds **1,000 competitors**, **6,704 competitions**, **4,139 venues** and **74 countries**.
- **Singles and doubles competitions** make up almost all competitions. Singles is slightly larger (about 3,700) than doubles (about 3,100), while mixed and mixed doubles are very small.
- The **USA has the most competitors** in the top-10 country chart (above 100), and most other top countries have between about 40 and 50.
- The top ranked doubles players by points are Katerina Siniakova (Czechia, 11,000), Harri Heliovaara (Finland, 10,540), Henry Patten (Great Britain, 10,540), Taylor Townsend (USA, 9,545) and Luisa Stefani (Brazil, 8,545).

Data observations

- "Neutral" appears in the country chart. It is not a real country; it is how the API labels athletes competing under a neutral flag.

- Several players share rank 1 in the top 5 table. This comes from combining records across men's and women's doubles rankings, so a rank filter by gender or tour is needed for a single clean ranking.

10. Challenges and Solutions

Challenge	Solution
API rate limits on the trial key	Added delays between calls and saved raw JSON so calls were not repeated
Nested JSON responses	Flattened the JSON with Pandas in `preprocessing.py`
Keeping foreign keys valid	Loaded parent tables first (Categories, Complexes, Competitors) before child tables
Reserved word `rank` in MySQL	Used backticks around `rank` in queries
Moving from local MySQL to the cloud	Dumped the data into a hosted MySQL service (Aiven) and kept credentials in environment variables or Streamlit secrets
Messy project folders	Separated raw JSON (`data/raw`) and generated CSV files (`data/csvfiles`)

11. Deployment

- Code is on GitHub: https://github.com/dhamayanthidevik/tennis_project
- The app is deployed on Streamlit Cloud.
- The database is hosted on a cloud MySQL service so the deployed app can reach it.
- Database credentials and the API key are stored as secrets and are never committed to the repository.

12. How to Run Locally

```
git clone https://github.com/dhamayanthidevik/tennis_project.git
cd tennis_project
pip install -r requirements.txt
# add database credentials to .streamlit/secrets.toml or environment
variables
streamlit run app/app.py
```

To rebuild the data from scratch: run `downloadjson.py`, `preprocessing.py`, `Build_table.py`, then execute `Create_query.sql` and run `loadmysql.py`.

13. Future Improvements

- Add singles rankings and live match data.
- Add year and week filters to every page for ranking history.
- Add indexes on frequently filtered columns and pagination for large tables.
- Add charts showing ranking movement over time for a selected competitor.
- Schedule automatic data refresh from the API.

14. Conclusion

This project covers the full data workflow: API data collection, cleaning, relational storage, SQL analysis and an interactive dashboard that anyone can open from a link. It meets the project's deliverables of a populated SQL database, API scripts, a Streamlit app and documentation, with all code and SQL queries uploaded to GitHub.